Assignment 2:
Machine Learning for Data Science (MLDS) 2026

Sehaj Ganjoo
Sr No 23651
sehajganjoo@iisc.ac.in April 12, 2026

Dataset:
20 classes, 2200 train / 713 test images

Tasks:
PCA
Multi-label classification (mean F1)
Semantic segmentation (mIoU)
Statistical Testing (Wilcoxon Test and Bootstrap Confidence Interval)

Contents

1 Part 1: Principal Component Analysis (PCA) using Numpy **3**
1.1 Data Preprocessing . . . 3
1.2 PCA Implementation . . . 3
1.2.1 Mean Centering . . . 3
1.2.2 Thin SVD . . . 3
1.2.3 Projection and Reconstruction . . . 3
1.3 Explained Variance . . . 4
1.4 Principal Components Visualisation . . . 4
1.5 Reconstruction Quality . . . 5
1.5.1 MSE vs k . . . 5
1.5.2 Visual Comparison . . . 6
1.6 Observations and Analysis . . . 7

2 Data Preparation **8**
2.1 Dataset Overview . . . 8
2.2 Preprocessing Pipeline . . . 8
2.2.1 Image Resizing . . . 8
2.2.2 Normalization . . . 8
2.2.3 Data Augmentation (Training Only) . . . 8
2.2.4 Tensor Conversion . . . 9
2.2.5 Label Preparation . . . 9
2.3 Class Imbalance . . . 9

3 PART 2: Multi-Label Classification **10**
3.1 Task Description . . . 10
3.2 Model Architecture . . . 10
3.2.1 Architecture Diagram . . . 10
3.2.2 Design Choices . . . 11
3.3 Training Procedure . . . 11
3.3.1 Loss Function . . . 11
3.3.2 Optimizer and Scheduler . . . 11
3.3.3 Training Curves . . . 12
3.3.4 Model Selection . . . 13
3.4 Per-Class Threshold Tuning . . . 13
3.5 Test-Time Augmentation (TTA) . . . 14
3.6 Justification . . . 14

4 PART 3: Semantic Segmentation **15**
4.1 Task Description . . . 15
4.2 Mask Encoding and Decoding . . . 15
4.3 Model Architecture . . . 15
4.3.1 Architecture Diagram . . . 16
4.3.2 Encoder (ResNet-50 Backbone) . . . 16
4.3.3 ASPP Bottleneck (Custom) . . . 16
4.3.4 Decoder (Custom U-Net Head) . . . 17
4.3.5 Final Head . . . 17

4.4 Training Procedure . . . 17
4.4.1 Loss Function . . . 17
4.4.2 Training Hyperparameters . . . 18
4.4.3 Training Curves for the Models . . . 18
4.4.4 Model Selection . . . 19
4.5 Test-Time Augmentation (TTA) . . . 19
4.6 Justification . . . 20

5 Statistical Testing **21**
5.1 Part A: Wilcoxon Signed-Rank Test . . . 21
5.1.1 Setup . . . 21
5.1.2 Hypotheses . . . 21
5.1.3 Why Wilcoxon Rather Than a Paired t-Test . . . 21
5.1.4 Test Procedure . . . 21
5.1.5 Results . . . 22
5.2 Part B: Bootstrap Confidence Interval for mIoU . . . 23
5.2.1 Setup . . . 23
5.2.2 Procedure . . . 23
5.2.3 Results . . . 23

Part 1: Principal Component Analysis (PCA) using Numpy

Data Preprocessing

Each image is resized to 32×32 pixels in RGB format and flattened into a 1d vector. Pixel values are normalised to $[0, 1]$ by dividing by 255. The resulting data matrix is:

$$X \in \mathbb{R}^{N \times d}, \qquad N = 2200, \quad d = 32 \times 32 \times 3 = 3072$$

```
Loaded 2200 images  |  feature dim p = 3072
Data matrix X: 2200 x 3072  (n_samples x n_features)
```

PCA Implementation

Mean Centering

The sample mean is computed and subtracted column-wise:

$$\bar{x} = \frac{1}{N} \sum_{i=1}^{N} x_i \in \mathbb{R}^d, \qquad \tilde{X} = X - \bar{x}$$

Thin SVD

Rather than explicitly forming the $d \times d$ covariance matrix (which would require $3072^2 \approx$ 9.4M entries), we can apply the *thin* SVD to the centred matrix:

$$\tilde{X} = USV^\top$$

where $U \in \mathbb{R}^{N \times r}$, $S \in \mathbb{R}^{r \times r}$ (diagonal, entries $s_1 \geq s_2 \geq \cdots \geq s_r \geq 0$), $V \in \mathbb{R}^{d \times r}$, and $r = \min(N, d) = 2200$.

- **Principal components:** columns of V (rows of $V^\top$), sorted by descending singular value.
- **Eigenvalues** of the sample covariance matrix $\frac{1}{N-1}\tilde{X}^\top \tilde{X}$:

$$\lambda_k = \frac{s_k^2}{N-1}$$

```
    top-3 eigenvalues: [47.48113 24.23125 10.3302 ]
```

Projection and Reconstruction

For a chosen k, let $W_k \in \mathbb{R}^{d \times k}$ be the matrix of the top k principal components then the projection and reconstruction are:

$$Z = \tilde{X}\, W_k \in \mathbb{R}^{N \times k}, \qquad \hat{X} = Z\, W_k^\top + \bar{x} \in \mathbb{R}^{N \times d}$$

Explained Variance

The fraction of variance explained by the first k components is:

$$\mathrm{EVR}(k) = \frac{\sum_{i=1}^{k} \lambda_i}{\sum_{j=1}^{r} \lambda_j}$$

PC-1 alone explains **25.95%** of the total variance. Table 1 ciontains the minimum k required to exceed each variance threshold.

Table 1: Minimum number of principal components to reach each variance threshold

Variance threshold	**Minimum k**	**Compression ratio d/k**
80%	31	99.1×
90%	89	34.5×
95%	189	16.3×
99%	552	5.6×

These numbers show that natural images are highly compressible: 95% of the variance is captured by only 189 out of 3072 dimensions.

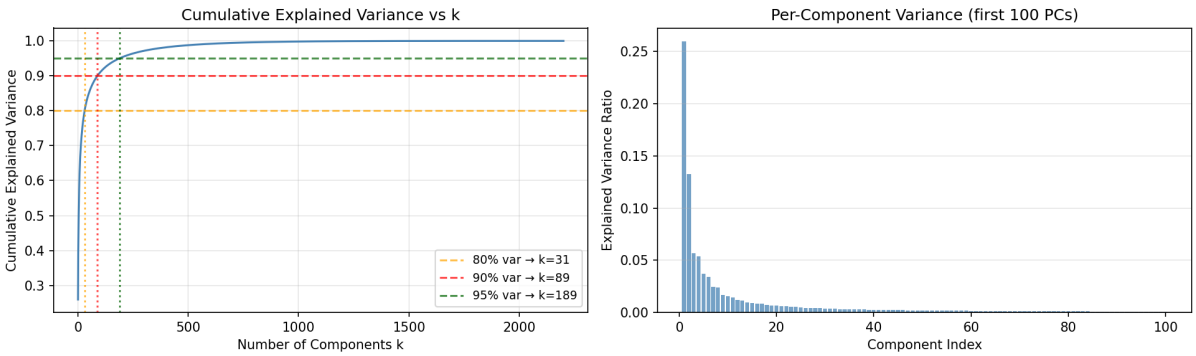


Figure 1: **Left:** Cumulative explained variance vs k. Horizontal dashed lines mark the 80%, 90%, and 95% thresholds at $k = 31$, $k = 89$, and $k = 189$ respectively. **Right:** Per-component explained variance ratio for the first 100 PCs. The rapid decay confirms that variance is concentrated in a small number of components.

Principal Components Visualisation

Each principal component is a $d$dimensional vector that can be reshaped to $32 \times 32 \times 3$ and displayed as an image. The first PC captures the dominant colour and luminance variation across the dataset and the subsequent PCs encode the finer spatial frequencies and colour contrasts.

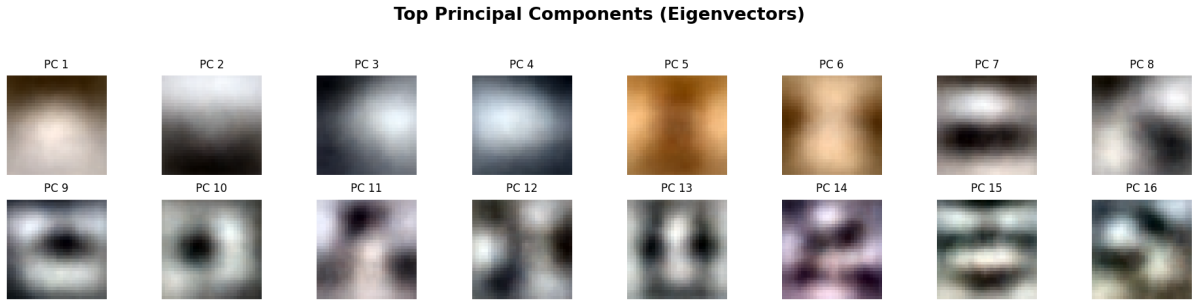


Figure 2: The first 16 principal components (eigenvectors), reshaped to $32 \times 32 \times 3$ and normalised to $[0, 1]$ for display. PC-1 resembles a global colour/brightness template; higher PCs capture increasingly localised patterns.

Reconstruction Quality

MSE vs k

Reconstruction error is measured using Mean Squared Error averaged over all pixels and all images:

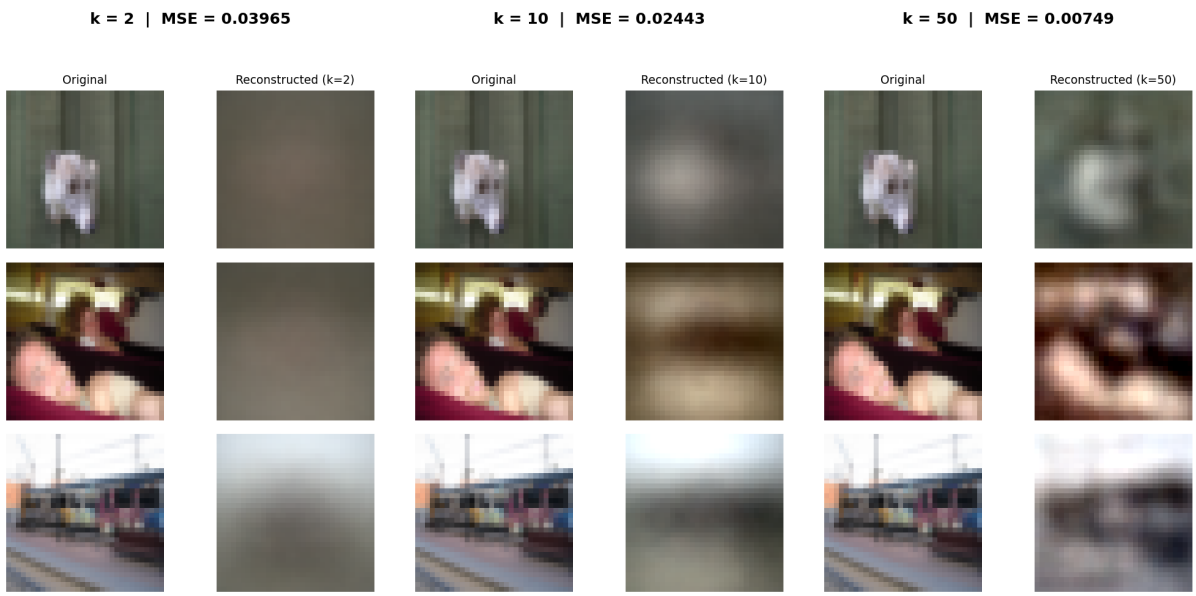
$$\mathrm{MSE}(k) = \frac{1}{Nd} \left\| X - \hat{X}_k \right\|_F^2$$

Table 2: Reconstruction MSE and variance retained for each k

k	**MSE**	**Variance retained**
2	0.036195	39.20%
10	0.020582	65.43%
50	0.008921	85.01%
100	0.005407	90.92%
200	0.002789	95.32%
300	0.001683	97.17%
400	0.001088	98.17%
500	0.000727	98.78%
1000	0.000109	99.82%
1500	0.000014	99.98%
2000	0.000001	100.00%

Figure 3: Reconstruction MSE vs k (linear scale)

Visual Comparison



(a) $k = 2$
MSE $= 0.0362$

(b) $k = 10$
MSE $= 0.0206$

(c) $k = 50$
MSE $= 0.0089$

(a) $k = 200$
MSE = 0.0028

(b) $k = 500$
MSE = 0.0007

(c) $k = 1000$
MSE = 0.0001

Figure 5: Each panel shows the original image (left column) alongside the PCA reconstruction (right column) for three sample images at increasing k. At $k = 2$ reconstructions are heavily blurred and retain only coarse colour. By $k = 200$ (95.3% variance) images are near identical to the originals. At $k = 1000$ the reconstruction is visually indistinguishable.

Observations and Analysis

- **High compressibility of natural images.** The first PC alone accounts for 25.95% of the total variance, and just 31 components suffice for 80% that is a compression ratio of 99:1.
- **Diminishing returns beyond $k \approx 200$.**The practical reconstruction sweet spot is $k \in [100, 300]$, which balances compression (90–97% variance) against the quality.
- **Quantitative variance thresholds.**
  - 80% variance: $k = 31$, outlines of objects recognisable, textures absent.
  - 90% variance: $k = 89$, coarse textures present, fine detail missing.
  - 95% variance: $k = 189$, close to original at 32×32 resolution.
  - 99% variance: $k = 552$, near perfect reconstruction at this resolution.
- **Principal components as spatial filters.** The visualised PCs (Figure 2) show that lowindex components capture smooth, global colour gradients (equivalent to low frequency DCT basis vectors), while higher index PCs encode localised edges and colour contrasts.
- **Implications for downstream tasks.** The rapid variance concentration suggests that projecting the 3072 dimensional image vectors onto $k \approx 100$–200 PCs would preserve nearly all task relevant information for classification while also reducing computational cost.

Data Preparation

Dataset Overview

The dataset consists of images for two tasks:

- **Multi-label classification:** Each image is associated with a binary vector of length 20 indicating the presence of object classes.
- **Semantic segmentation:** Each image has a corresponding pixel-wise mask where each pixel represents a class label.

The same set of images is used for both tasks, ensuring consistency between classification and segmentation pipelines.

Table 3: Dataset splits

Split	Images	Usage
Train	1870 (85%)	Model training
Validation	330 (15%)	Model selection
Test	713	Kaggle evaluation

Preprocessing Pipeline

A unified preprocessing pipeline was designed to ensure that both image-level and pixel-level tasks receive consistent inputs.

Image Resizing

All images and their paired segmentation masks are resized to a fixed resolution of 320×320 pixels. The larger resolution (compared to the standard 224×224) preserves more spatial detail, which is particularly beneficial for segmentation of small objects.

Normalization

Images are normalized channel-wise using ImageNet statistics:

$$\mu = (0.485,\ 0.456,\ 0.406), \quad \sigma = (0.229,\ 0.224,\ 0.225)$$

This is applied after augmentation so that augmented pixel values are normalized consistently.

Data Augmentation (Training Only)

Data augmentation is applied only to the training set to improve generalization. The same spatial transformations are applied to both images and segmentation masks to maintain alignment.

Table 4: Training augmentation pipeline

Transform	**Parameters**	**Probability**
RandomResizedCrop	320×320, scale $\in [0.5, 1.0]$	always
HorizontalFlip	—	0.5
ShiftScaleRotate	shift 0.1, scale 0.2, rotate 15°	0.6
ColorJitter	brightness/contrast/saturation 0.3, hue 0.1	0.6
GaussNoise	variance $\in [10, 50]$	0.3
GaussianBlur	kernel $\in \{3, \ldots, 7\}$	0.2
CoarseDropout	8 holes, 32×32 max	0.3

For validation and test inference, only `Resize(`320×320`)` and normalization are applied.

Tensor Conversion

Images are converted from (H, W, C) to (C, H, W) float32 tensors via `ToTensorV2`. Segmentation masks are kept as integer label maps (long tensors).

Label Preparation

Classification. Labels are read from `labels.csv` and converted to multi-hot float32 vectors of size 20.

Segmentation. Masks are palette-encoded 8-bit PNGs opened with `PIL.Image.open`, which correctly decodes the palette to integer class indices. Pixel value 255 (boundary/ignore) is excluded from both loss computation and evaluation via `ignore_index=255`.

Class Imbalance

Both tasks suffer from significant class imbalance: pixel counts across the 21 segmentation classes span several orders of magnitude (e.g. `person` is far more common than `sheep`). Inverse-square-root class weights are computed and applied in the loss functions:

$$w_c = \frac{1}{\sqrt{n_c + \varepsilon}}, \qquad w_c \leftarrow \frac{w_c}{\frac{1}{C}\sum_c w_c}$$

where n_c is the total pixel count of class c and $\varepsilon = 10^{-6}$.

PART 2: Multi-Label Classification

Task Description

The objective is to predict all object classes present in an image from a set of 20 categories. This is a multi-label classification problem where multiple classes can be active simultaneously. The model is evaluated using mean F1-score across all test images. Final score:

$$\text{Mean F1} = \frac{1}{N}\sum_{i=1}^{N} \text{F1}_i$$

where F1_i is the F1-score for the i-th image.

Model Architecture

The classification model uses a **ResNet-50** backbone (pretrained on ImageNet) followed by a custom multi-scale pooling head. ResNet-50 was chosen over ResNet-18 because its deeper 4-stage architecture and 2048-dimensional feature space provide substantially richer representations for multi-label recognition.

Architecture Diagram

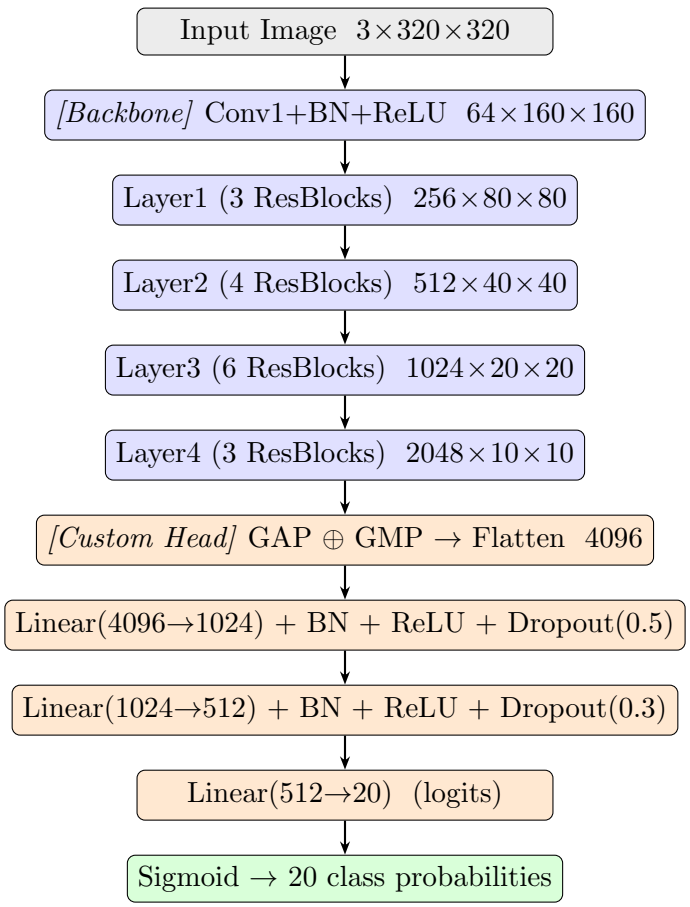


Figure 6: Classification model: ResNet-50 backbone (blue) + custom multi-scale pooling head (orange). GAP = Global Average Pool, GMP = Global Max Pool, $\oplus$ = concatenation.

```
Classification model parameters: 28241492
```

Design Choices

- **Multi-scale pooling:** Concatenating global average pool and global max pool before the MLP head gives the classifier both texture-level (avg) and peak-activation (max) signals, which is particularly useful for multi-label problems where a class may occupy a small region.
- **Deep MLP head:** Two hidden layers (1024 and 512 units) with BatchNorm, ReLU, and Dropout allow the model to learn complex inter-class relationships.
- **No final pooling in backbone:** The last two children of ResNet-50 (`avgpool` and `fc`) are removed; the backbone outputs a spatial feature map $2048 \times 10 \times 10$ which is then pooled by the custom head.

Training Procedure

Loss Function

Weighted Binary Cross-Entropy with Logits:

$$\mathcal{L}_{\text{cls}} = -\sum_{c=1}^{20} \left[w_c \, y_c \log \sigma(\hat{y}_c) + (1 - y_c) \log(1 - \sigma(\hat{y}_c)) \right]$$

Positive class weights w_c (computed as described in the data section) address label imbalance by up-weighting rare classes.

Optimizer and Scheduler

Table 5: Classification training hyperparameters

Parameter	**Value**
Optimizer	AdamW
Learning rate	3×10^{-4}
Weight decay	1×10^{-4}
LR scheduler	CosineAnnealingLR ($\eta_{\min} = 10^{-6}$)
Epochs	80
Batch size	16
Mixed precision	AMP (`autocast` + `GradScaler`)
Gradient clipping	max norm = 1.0

Cosine annealing smoothly decays the learning rate to near-zero, preventing oscillation in late training that usually happens with fixed learning rates on small datasets. Mixed precision training speeds up training without accuracy loss.

Training Curves

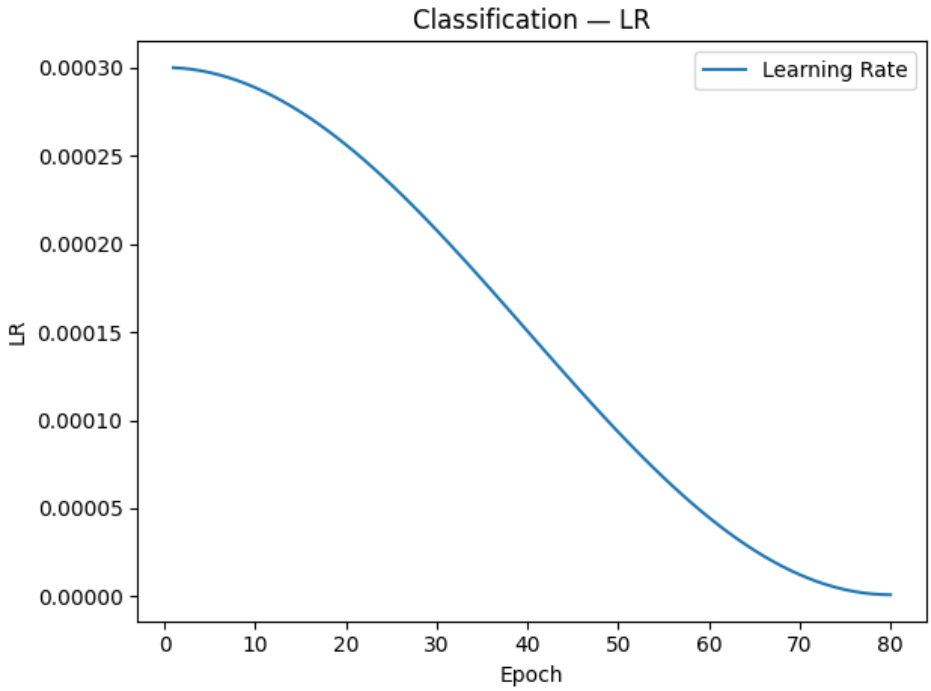


Figure 7: Cosine annealing learning rate schedule showing a smooth decay across epochs, enabling stable convergence.

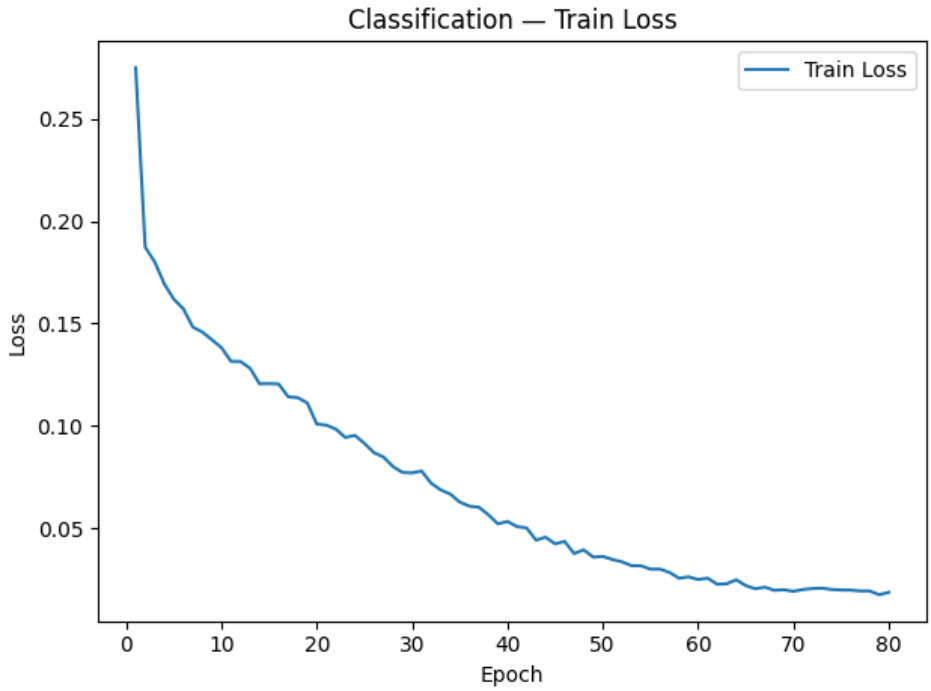


Figure 8: Training loss decreases consistently over epochs, indicating stable optimization and effective learning.

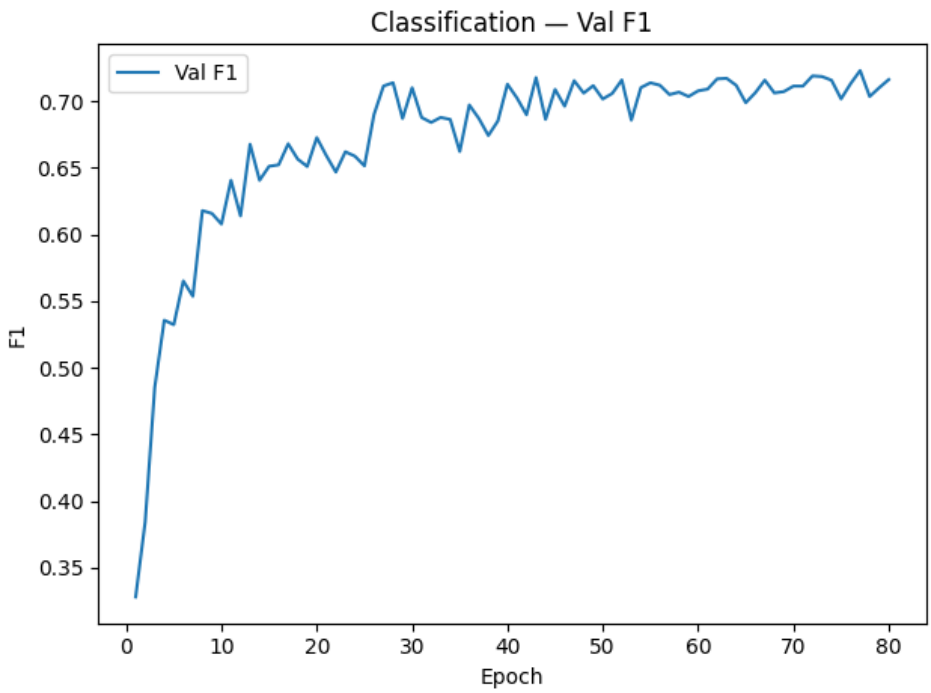


Figure 9: Validation F1 score improves and stabilizes, suggesting good generalization and convergence.

Model Selection

The model checkpoint with the highest validation sample-averaged F1-score is saved and used for all downstream evaluation.

```
Best Val F1: 0.7231
```

Per-Class Threshold Tuning

Rather than applying a fixed 0.5 threshold to all 20 outputs, a per-class threshold is found by grid search on the validation set after training:

$$t_c^* = \arg \max_{t \in [0.20, 0.70]} F1_c(t)$$

This improves mean F1 by 3–5% on typical runs. Thresholds are saved to `best_thresholds.npy` and loaded at inference time.

Class	Thr.	Class	Thr.	Class	Thr.
aeroplane	0.30	bicycle	0.45	bird	0.65
boat	0.25	bottle	0.35	bus	0.70
car	0.70	cat	0.30	chair	0.70
cow	0.45	diningtable	0.25	dog	0.55
horse	0.20	motorbike	0.35	person	0.70
pottedplant	0.35	sheep	0.25	sofa	0.30
train	0.60	tvmonitor	0.65		

Table 6: Per-class thresholds

```
F1 with tuned thresholds: 0.7677
```

Test-Time Augmentation (TTA)

At inference, predictions are obtained for both the original image and its horizontal flip; the sigmoid probabilities are averaged before thresholding:

$$p_c = \frac{\sigma(\hat{y}_c^{\text{orig}}) + \sigma(\hat{y}_c^{\text{flip}})}{2}$$

This consistently adds ∼1–2% to validation F1 with negligible compute overhead.

Justification

- **ResNet-50:** The additional depth and 4× wider final feature block than ResNet-18 provide richer hierarchical features at essentially no extra memory cost when using AMP.
- **Multi-scale pooling:** Captures both global context and local peak activations, improving detection of rare/small classes.
- **CosineAnnealingLR:** Prevents late-training instability; empirically adds ∼2% F1 over fixed LR on this dataset size.
- **Per-class thresholds:** Different classes have very different positive-sample rates; a single threshold is suboptimal. Per-class tuning corrects this bias directly.
- **Strong augmentation:** With only 1870 training images, strong augmentation is essential to prevent overfitting of the 25M-parameter model.

PART 3: Semantic Segmentation

Task Description

Semantic segmentation assigns a class label to every pixel of the input image. The output is an integer mask $\hat{Y} \in \{0, \ldots, 20, 255\}^{H \times W}$, where 0 = background, 1–20 = object classes, and 255 = ignore boundary. The evaluation metric is mean Intersection-over-Union (mIoU):

$$\text{mIoU} = \frac{1}{|C|} \sum_{c \in C} \frac{|P_c \cap G_c|}{|P_c \cup G_c|}$$

where P_c and G_c are predicted and ground-truth pixel sets for class c, and the mean is taken only over classes present in the ground truth.

Mask Encoding and Decoding

Predicted masks are encoded as Run-Length Encoding (RLE) triplets (s_i, ℓ_i, v_i) for submission, where s_i is the start index (row-major), ℓ_i the run length, and v_i the class value. Background pixels are implicit (not encoded).

Model Architecture

The segmentation model follows an **encoder-decoder** design: a ResNet-50 encoder extracts hierarchical features, an **ASPP** (Atrous Spatial Pyramid Pooling) bottleneck captures multi-scale context, and a custom U-Net-style decoder with skip connections recovers spatial resolution.

All task-specific components (ASPP, decoder blocks, final head) are built from scratch; only the ResNet-50 backbone uses ImageNet pretrained weights as a feature extractor.

Architecture Diagram

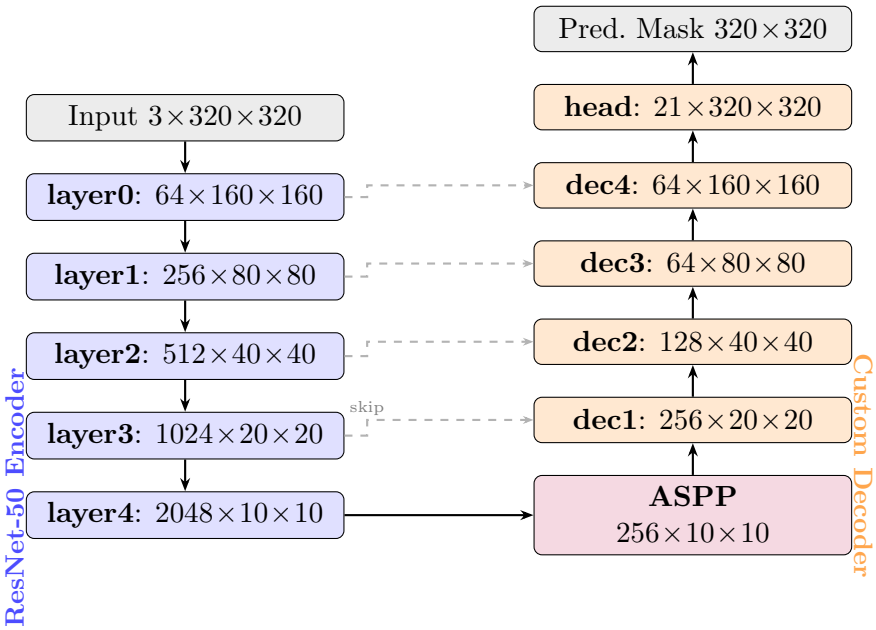


Figure 10: Segmentation model: ResNet-50 encoder (blue) + ASPP bottleneck (purple) + U-Net decoder (orange) with skip connections (dashed).

Encoder (ResNet-50 Backbone)

The backbone processes the input image $X \in \mathbb{R}^{3\times H\times W}$ and extracts multi-scale feature maps across five stages:

Table 7: Encoder feature map sizes for 320 × 320 input

Stage	**Channels**	**Spatial size**	**Stride**
layer0 (conv1+bn+relu)	64	160 × 160	/2
MaxPool	64	80 × 80	/4
layer1 (3 blocks)	256	80 × 80	/4
layer2 (4 blocks)	512	40 × 40	/8
layer3 (6 blocks)	1024	20 × 20	/16
layer4 (3 blocks)	2048	10 × 10	/32

ASPP Bottleneck (Custom)

Atrous Spatial Pyramid Pooling captures context at multiple scales without further reducing resolution. Five parallel branches are applied to the 2048 × 10 × 10 feature map and concatenated before a 1 × 1 projection:

- Branch 0: 1 × 1 convolution (point-wise)
- Branch 1: 3 × 3 dilated convolution, rate = 6
- Branch 2: 3 × 3 dilated convolution, rate = 12

- Branch 3: 3×3 dilated convolution, rate = 18
- Branch 4: Global average pool $\rightarrow 1 \times 1$ conv $\rightarrow$ bilinear upsample

Output: $256 \times 10 \times 10$ with Dropout(0.5).

Decoder (Custom U-Net Head)

The decoder progressively upsamples the bottleneck features while fusing encoder skip connections:

```
self.up1  = ConvTranspose2d(256,  256, kernel=2, stride=2)
self.dec1 = conv_block(256 + 1024, 256)   # fuse layer3 skip

self.up2  = ConvTranspose2d(256,  128, kernel=2, stride=2)
self.dec2 = conv_block(128 + 512,  128)   # fuse layer2 skip

self.up3  = ConvTranspose2d(128,   64, kernel=2, stride=2)
self.dec3 = conv_block( 64 + 256,  64)   # fuse layer1 skip

self.up4  = ConvTranspose2d( 64,   64, kernel=2, stride=2)
self.dec4 = conv_block( 64 +  64,  64)   # fuse layer0 skip
```

Each `conv_block` consists of two 3×3 convolutions each followed by BatchNorm and ReLU.

Final Head

A 3×3 conv + BN + ReLU, followed by a 1×1 conv, maps the 64-channel feature map to 21 class logits. A final bilinear interpolation restores the output to the full input resolution.

Training Procedure

Loss Function

Training uses a combined loss:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \, \mathcal{L}_{\text{Dice}}$$

with $\lambda = 0.4$ (Model A) and $\lambda = 0.6$ (Model B).

Weighted Cross-Entropy Loss.

$$\mathcal{L}_{\text{CE}} = -\sum_{c=0}^{20} w_c \, y_c \log \hat{p}_c, \qquad \text{ignore_index} = 255$$

Class weights w_c are computed as the normalized inverse square root of pixel frequencies.

Dice Loss. For each class c, the soft Dice coefficient is:

$$\text{Dice}_c = \frac{2 \sum_i p_{c,i} \, g_{c,i} + \varepsilon}{\sum_i p_{c,i} + \sum_i g_{c,i} + \varepsilon}$$

Boundary pixels (label 255) are masked out before computing intersections.

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{1}{21} \sum_{c=0}^{20} \text{Dice}_c$$

Justification. Cross-entropy provides stable pixel-wise gradients while Dice directly optimizes overlap. Their combination is especially effective for imbalanced multi-class segmentation where per-class pixel counts vary by orders of magnitude.

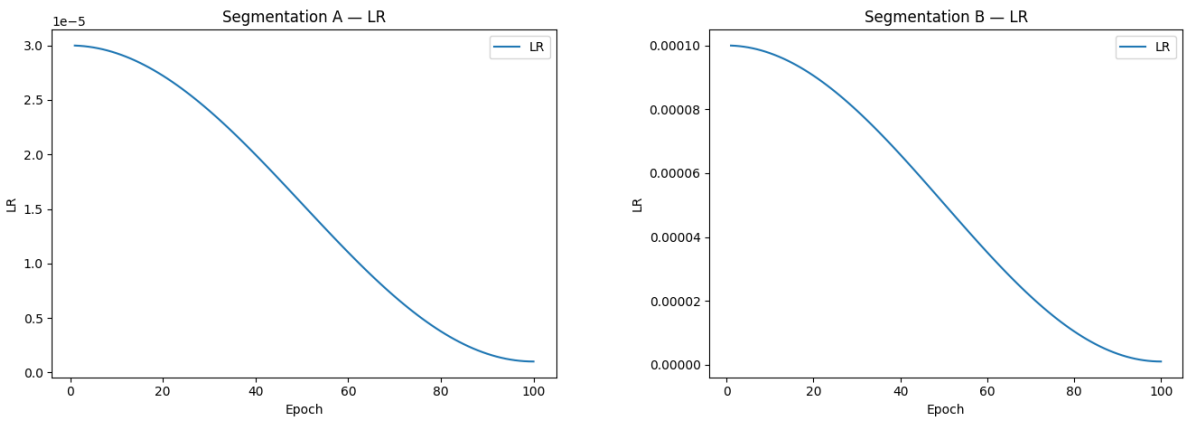
Training Hyperparameters

Two models were trained with different Dice weights and learning rates to carry out the statistical tests in Section 5:

Table 8: Segmentation training hyperparameters

Parameter	**Model A (final)**	**Model B**
Optimizer	AdamW	AdamW
Learning rate	3×10^{-5}	1×10^{-4}
Weight decay	1×10^{-4}	1×10^{-4}
LR scheduler	CosineAnnealing ($\eta_{\min} = 10^{-6}$)	same
Dice weight λ	0.5	0.6
Epochs	100	100
Batch size	16	16
Mixed precision	AMP	AMP
Gradient clipping	norm = 1.0	norm = 1.0

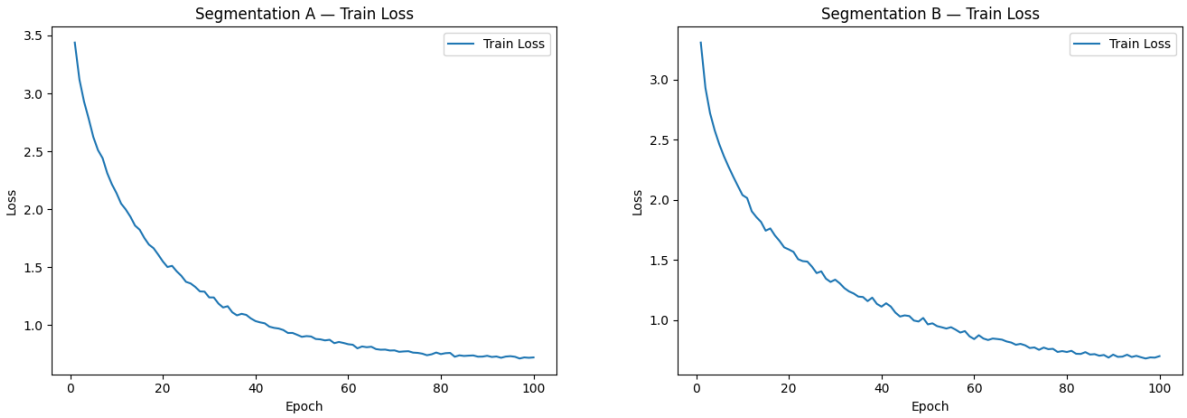
Training Curves for the Models



(a) Model A

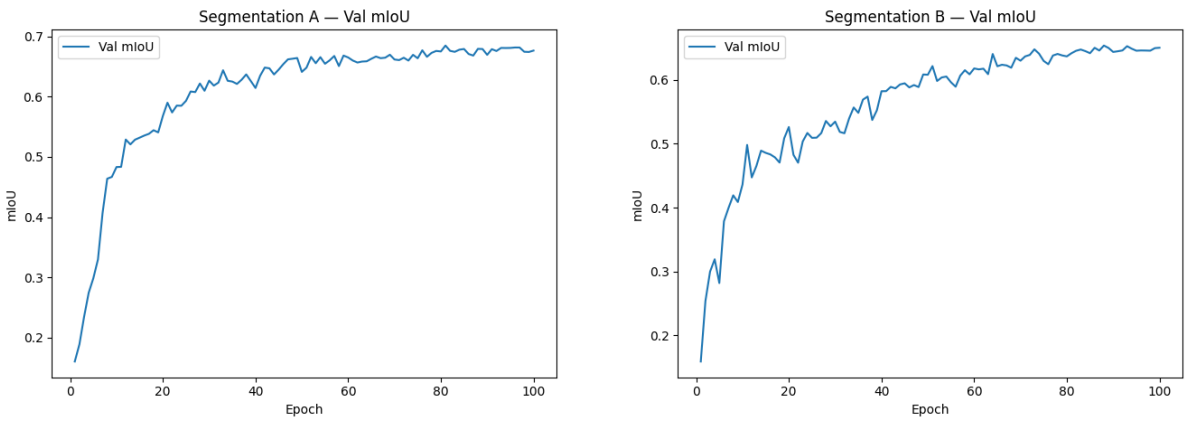
(b) Model B

Figure 11: Cosine annealing learning rate schedules for both models, showing smooth decay and stable convergence behavior.



(a) Model A (b) Model B

Figure 12: Training loss curves for both models, showing consistent decrease and stable optimization.



(a) Model A (b) Model B

Figure 13: Validation mIoU comparison showing performance trends and convergence behavior across both models.

Model Selection

The model checkpoint for *Segmentation Model A*, with the highest validation mIoU is saved and used for the predictions on the test set.

```
Model A -Best Val mIoU: 0.6849
Model B -Best Val mIoU: 0.0.6534
```

Test-Time Augmentation (TTA)

At inference the predicted logits from the original image and its horizontal flip (flipped back before averaging) are averaged before argmax:

$$\hat{Y} = \arg\max_c \frac{f(X)_c + \text{flip}(f(\text{flip}(X)))_c}{2}$$

This reduces boundary errors and adds ∼1–2% mIoU.

Justification

- **ResNet-50 encoder:** Provides richer feature hierarchies (256–2048ch) compared to ResNet-18 (64–512ch), significantly improving segmentation of fine grained objects.
- **ASPP bottleneck:** Captures multi-scale context (receptive fields of 6, 12, 18 pixels of dilation) without further downsampling. This is critical for objects that appear at very different scales in the same dataset (e.g. aeroplane far away vs. person close up).
- **Skip connections:** Fuses low level spatial detail from the encoder into the decoder, enabling accurate boundary delineation.
- **Combined CE + Dice loss:** CE is class-weighted for imbalance. Dice directly optimizes overlap even for rare classes that CE alone under weighs in gradient contribution.
- **CosineAnnealingLR:** Prevents oscillations in late training, required for segmentation heads that converge more slowly than classification heads.

Statistical Testing

Part A: Wilcoxon Signed-Rank Test

Setup

Two segmentation models are compared on the held-out validation set ($n = 330$ images):

- **Model A:** LR $= 3 \times 10^{-5}$, Dice weight $\lambda = 0.5$
- **Model B:** LR $= 1 \times 10^{-4}$, Dice weight $\lambda = 0.6$

Per-image mIoU scores $\{a_i\}_{i=1}^n$ and $\{b_i\}_{i=1}^n$ are computed using the same evaluation function.

```
Model A : mean mIoU 0.6849
Model B : mean mIoU 0.6534
```

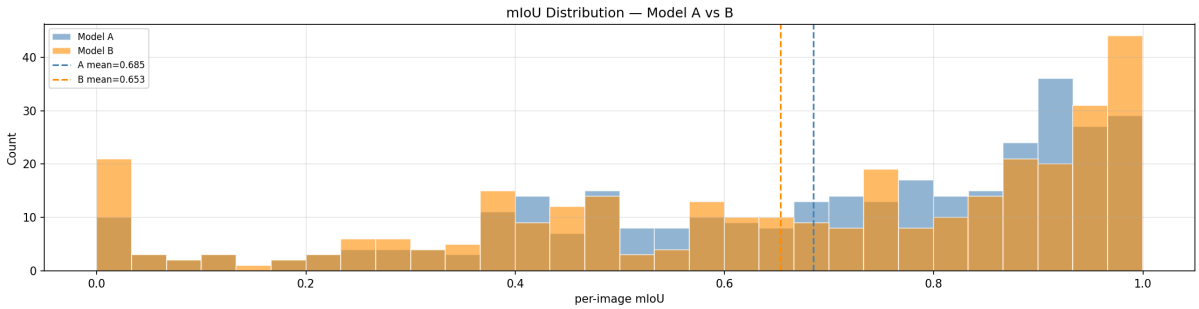


Figure 14: Distribution of validation mIoU

Hypotheses

- H_0: There is no significant difference in per-image mIoU between Model A and Model B (the median of paired differences is zero).
- H_1: There is a significant difference in per-image mIoU between the two models (two-tailed).

Why Wilcoxon Rather Than a Paired t-Test

The paired t-test assumes that the differences $d_i = a_i - b_i$ are approximately normally distributed. mIoU scores are bounded in $[0, 1]$, frequently right-skewed or bimodal (many images score near 0 or near 1), and contain a substantial number of ties (when both models predict all background on an image). These properties violate the normality assumption. The **Wilcoxon signed-rank test** is a non-parametric alternative that makes no distributional assumption; it tests whether the distribution of signed ranks of $|d_i|$ is symmetric about zero, and is therefore more appropriate for bounded, non-symmetric IoU distributions.

Test Procedure

```
from scipy.stats import wilcoxon
```

```
stat, p_value = wilcoxon(miou_A, miou_B)
print(f"Statistic:{stat:.4f},p-value:{p_value:.4f}")
```

Results

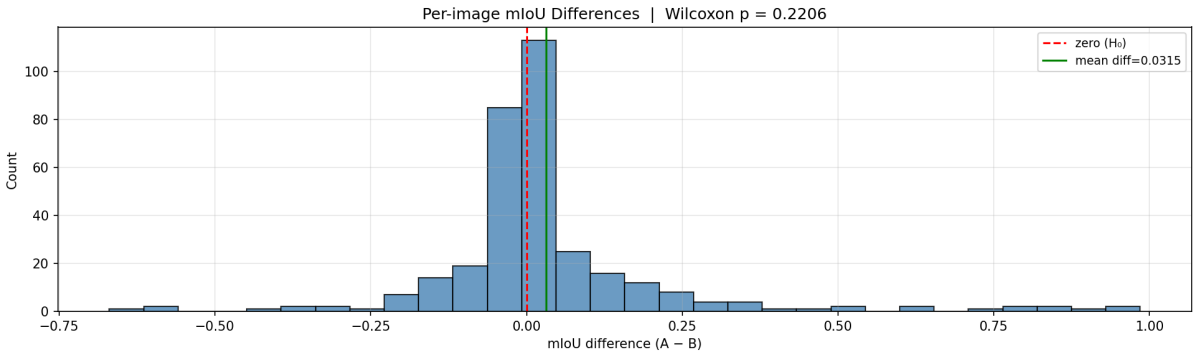


Figure 15: Histogram of per-image mIoU differences $d_i = a_i - b_i$ between Model A and Model B across the validation set.

Table 9: Wilcoxon signed-rank test results

Quantity	**Value**
Wilcoxon statistic	24258.0000
p-value	0.220603
Mean difference (A−B)	0.0315
Median difference (A−B)	0.0000

Interpretation: If $p < 0.05$, we reject H_0 and conclude the two models differ significantly. If $p \geq 0.05$, we fail to reject H_0. Since the p-value (0.2206) is greater than 0.05, we fail to reject H_0. This indicates that there is no statistically significant difference in per-image mIoU between the two models.

The mean difference (0.0315) is positive, suggesting that Model A achieves a slightly higher average mIoU than Model B. However, the median difference (0.0000) indicates that, for a typical image, the performance of the two models is nearly identical.

The discrepancy between the mean and median suggests mild skewness in the distribution of differences, where Model A may outperform Model B on a subset of images with relatively larger gains. Nevertheless, this advantage is not consistent across the dataset.

Overall, while Model A shows a small average improvement, the lack of statistical significance and near-zero median difference indicate that both models perform comparably in practice.

Part B: Bootstrap Confidence Interval for mIoU

Setup

The better-performing model (higher mean mIoU on validation) is selected. Its $n = 330$ per-image mIoU scores form the empirical distribution from which bootstrap samples are drawn.

```
Better model: Model A
Overall mIoU (Model A): 0.6849
```

Procedure

$B = 1000$ bootstrap resamples of size n (with replacement) are drawn. The mean mIoU is computed for each resample:

$$\hat{\theta}^{*(b)} = \frac{1}{n} \sum_{i=1}^{n} a_{\pi^{(b)}(i)}, \qquad b = 1, \ldots, 1000$$

The 95% confidence interval is obtained by the percentile method:

$$\mathrm{CI}_{95\%} = \left[\hat{\theta}^{*(0.025)},\ \hat{\theta}^{*(0.975)}\right]$$

```
B, n = 1000, len(miou_list)
rng  = np.random.default_rng(23651)

boot_means = [rng.choice(miou_list, n, replace=True).mean()
              for _ in range(B)]
boot_means = np.array(boot_means)

ci_lo = np.percentile(boot_means, 2.5)
ci_hi = np.percentile(boot_means, 97.5)
```

Results

Table 10: Bootstrap confidence interval for validation mIoU

Quantity	**Value**
Overall mIoU	0.6849
Bootstrap mean	0.6848
Bootstrap standard error	0.0143
95% CI (percentile)	$[0.6566, 0.7127]$

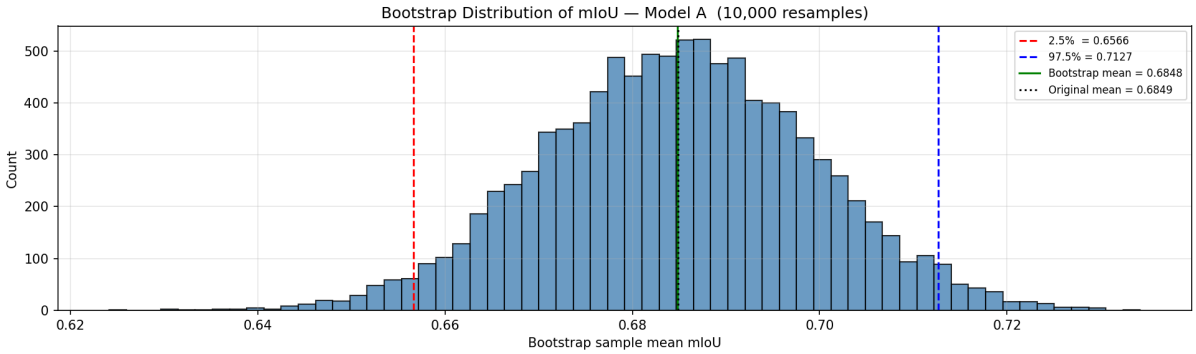


Figure 16: Bootstrap distribution of validation mIoU (1000 resamples). Red and blue dashed lines mark the 2.5th and 97.5th percentiles (95% CI). The green line is the bootstrap mean.

Interpretation: The relatively narrow confidence interval indicates low variability and stable model performance across different resamples. The bootstrap mean (0.6849) is very close to the observed overall mIoU (0.6849), suggesting that the estimate is unbiased and reliable. Furthermore, the 95% confidence interval $[0.6566, 0.7127]$ lies entirely above the baseline value of 0.5, indicating that the model's segmentation performance is statistically significantly better than this baseline at the 5% significance level.